圧倒的キャラクター数×カスタマイズ数を ゲームエンジンで実現 ～『鉄拳8』キャラクターグラフィックス事例～

株式会社バンダイナムコスタジオ

冨澤 茂樹

TEKKEN™8 & ©Bandai Namco Entertainment Inc.
発売元：株式会社バンダイナムコエンターテインメント

イントロダクション

自己紹介

冨澤 茂樹

株式会社バンダイナムコスタジオ

2000年旧ナムコに同業他社より中途入社

内製ミドルウェア初代NUライブラリの立ち上げ開発・保守を行う

その後いくつかのプロジェクトにて60fps化を行う

TEKKEN™ MOBILE プロジェクトにてグラフィックス担当(CEDEC2018)

『鉄拳8』にて描画エンジニアパートを担当

描画エンジニアパート

- 最大 7.5 名
  - ビルドエンジニア 1 名、ゲームやネットワークと掛持ちも含む
- ゲームエンジンを採用しても描画エンジニアの役割は多い
  - VA のやりたいグラフィックスを実現する
  - 企画のやりたい仕様を実現する
    - Unreal Engine の深い知識、Editor Utility Widget からエンジン改造まで
  - 負荷計測 … デイリー自動負荷計測の仕組みを作成
    - ビルドエンジニアのおかげ
  - プロファイラーで負荷を下げる方法を探す
  - Device Removed の原因究明
    - GPU の深い知識を発揮

Unreal Engine 採用

- 前作が採用したからではなく新たに評価
- PlayStation5, Xbox Series X|S のサポート
- メジャーバージョンアップ 4 → 5 の懸念はあった

Unreal Engine 5 について

- 新機能は利用せず
- Lumen
  - レイトレーシングとラスタライズレンダリングの混在が不可だった
- Nanite
  - メッシュシェーダー的な手法は良い
  - 狭いフィールド内なので頂点ストリーミング向きではなかった
- バーチャルテクスチャー
- バーチャルシャドウマップ
  - 狭いフィールド内なのでストリーミング向きではなかった
- Niagara は使っている

本日の発表内容

- 体格調整について
  - データフロー
  - ランタイム実装
  - メッシュ変形の実装
  - 問題点
- その他の事例
  - テクスチャーベイク
  - しわ自動計算
- まとめ

体格調整について

体格調整とは

- キャラクター共通の標準体型からキャラクター固有体型へランタイムで変形させる
- キャラクター共通の標準体型で作った衣装を使いまわせる
- アセット数がキャラクター数×カスタマイズ数より圧倒的に減る
- 詳しくは
- 『鉄拳8』 カスタマイズキャラクターのための多重リグシステム ～ 複雑な筋肉表現と大量生産の両立
  - 8月21日(水) 13:40 ～ 14:40 第3会場

体格調整をご紹介するのは

- キャラクターカスタマイズにおける
  「圧倒的キャラクター数×カスタマイズ数」
  の制作コストを軽減する事例
- エンジン改造せずに実装できた事例
- メッシュ変形は実装できる
- C++ による実装 (BP 使いのかたごめんなさい…)
- エンジンのヘッダファイルを見れば大抵のことはできる
- ただ意外とネット上に情報が少ない

体格調整の例

標準体型

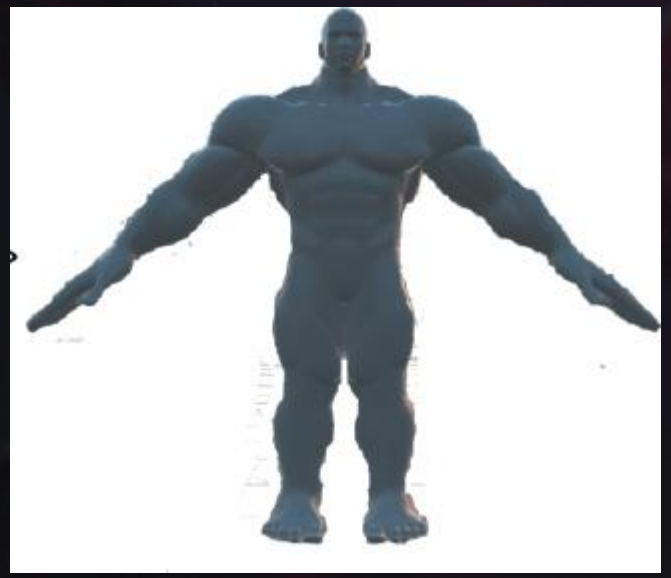
キャラ固有体型

体格調整の手順

1. 標準体型から体格調整用のスケルトンとウェイトを使用して
   キャラクター固有体型へ変形する
2. ゲームアニメーション用のスケルトンとバインド
3. ゲームアニメーション用にウェイトを差替える

1

2, 3



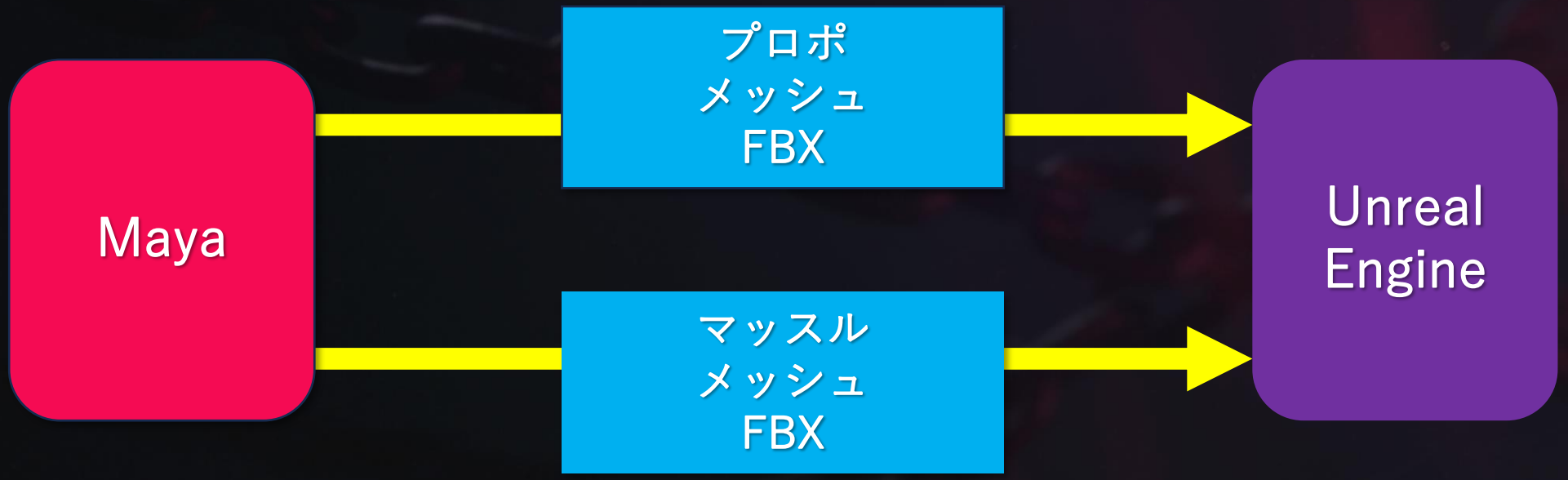
用語を整理

- プロポーションメッシュ
  - 標準体型のメッシュ
  - プロポメッシュと略します
- プロポーションリグ
  - 標準体型からキャラ固有体型に変形するためのスケルトン
  - プロポリグと略します
- マッスルメッシュ
  - キャラ固有体型のメッシュ
- マッスルリグ
  - 実際にゲーム中バトルでアニメーションに使うスケルトン

データフロー

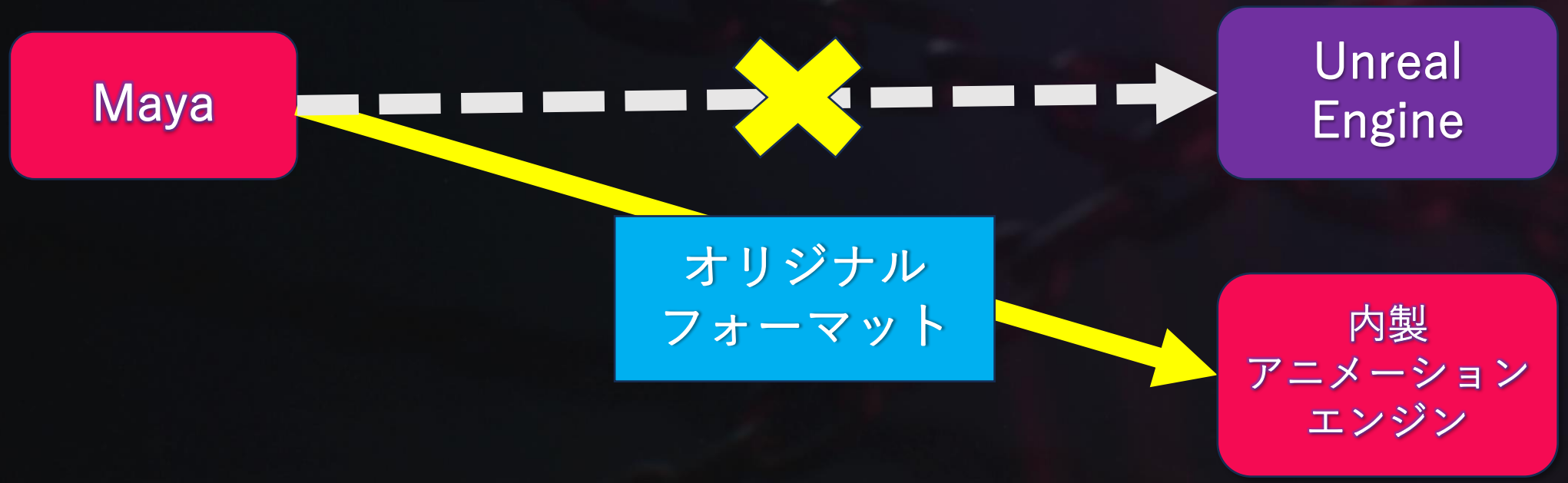
Maya からインポート

- メッシュは Maya から FBX を経由して UE へ



アニメーションデータ

- アニメーションデータは内製のアニメーションエンジンへ
- 詳しくは
- 『鉄拳8』アニメーション技術 ～鉄拳７からの進化
  - 8月21日(水) 16:40 ～ 17:05 第3会場



それぞれのメッシュの役割

- プロポメッシュ
  - 標準体型の頂点情報
  - プロポリグのバインド情報とウェイトを持つ
- マッスルメッシュ
  - マッスルリグのバインド情報とウェイトを参照する
  - position, normal, UV 等は未使用
- 余分な情報も多いがデータフローを統一するため

ランタイム実装

体格調整のマトリクス列を作る

- アニメーションエンジンからプロポリグの
  標準体型とキャラ固有体型のマトリクス列を取り出す

体格調整のマトリクス列を作る

- 体格調整変換行列を求める
- 法線変換行列も求めておく

$$\left(M_{transform}\right)_{[0\cdots n-1]} = \left(M_{before}\right)^{-1}_{[0\cdots n-1]}\left(M_{after}\right)_{[0\cdots n-1]}$$

$$\left(M_{normal}\right)_{[0\cdots n-1]} = \left(M_{transform}\right)^{-T}_{[0\cdots n-1]}$$

マトリクス列を並べ替える

- アニメーションエンジンとスケルタルメッシュでボーン順が一致しているとは限らない
- アニメーションエンジンからボーン名を取り出す
- スケルタルメッシュにボーン名が配列の何番目かを問い合わせる

```
const FReferenceSkeleton& RefSkeleton = pSkeletalMesh->GetRefSkeleton();

int32 BoneIndex = RefSkeleton.FindBoneIndex(BoneName);
```

手順1：標準体型から固有体型へ変形

- 頂点データは FSkeletalMeshRenderData クラスが持っている
- FSkeletalMeshRenderData クラスは USkeletalMesh クラスから取得・生成ができる
- プロポメッシュの頂点データを変形&コピー
- メッシュ変形について詳しくは後述

```
// 取得.
FSkeletalMeshRenderData* pRenderData =
                pSkeletalMesh->GetResourceForRendering();

// 生成.
pSkeletalMesh->AllocateResourceForRendering();
```

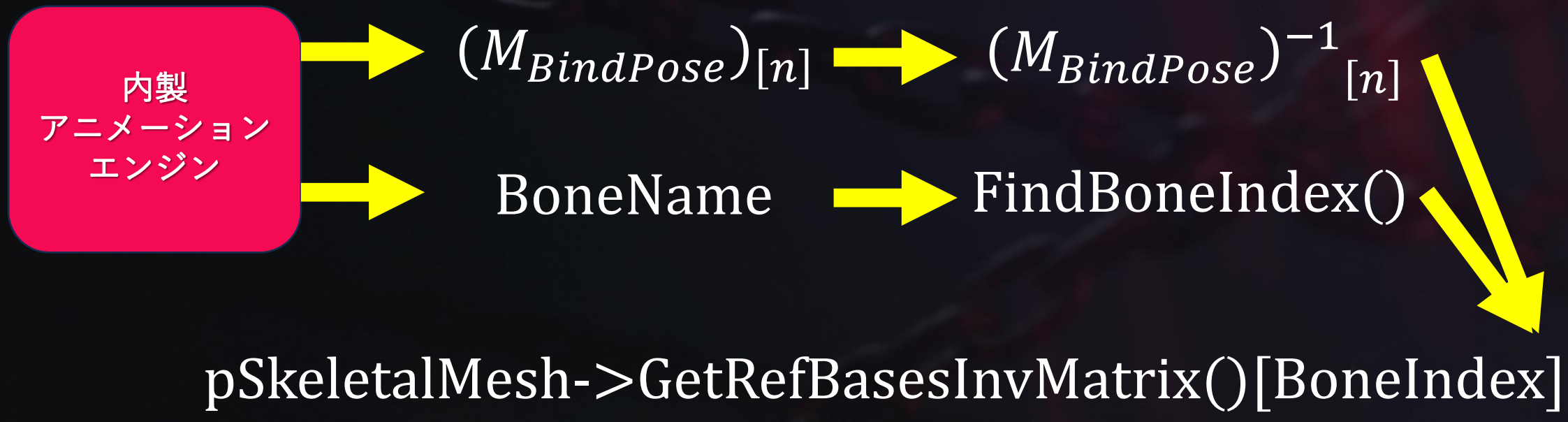
変形用のマトリクス

- ボーンインデックスとウェイトでマトリクスを生成
- インフルエンスが 4 の場合は下記のようになる
- Normal 変換マトリクスも同様に作る

$$
\begin{aligned}
M_{TRANSFORM} = \mathrm{Weight0}\left(M_{transform}\right)_{[BoneIndex0]} \\
+\mathrm{Weight1}\left(M_{transform}\right)_{[BoneIndex1]} \\
+\mathrm{Weight2}\left(M_{transform}\right)_{[BoneIndex2]} \\
+\mathrm{Weight3}\left(M_{transform}\right)_{[BoneIndex3]}
\end{aligned}
$$

手順2：マッスルリグにバインド

- アニメーションエンジンからバインドポーズのマトリクス列を取り出し逆行列にして USkeletalMesh::GetRefBasesInvMatrix() に格納する
- もちろんボーン名による並べ替えは行う



手順3：マッスルリグのウェイトに差替える

- マッスルメッシュのウェイトデータ FSkeletalMeshLODRenderData::SkinWeightVertexBuffer をコピーする
- ただしセクション(FSkelMeshRenderSection)の影響は考慮する
- 詳しくはこの後のメッシュ変形の章で

メッシュ変形の実装

FSkeletalMeshRenderData クラス

- FSkeletalMeshRenderData クラスを書き換えることがメッシュ変形
- 実データは FSkeletalMeshLODRenderData クラスが持っている
- FSkeletalMeshRenderData は FSkeletalMeshLODRenderData の配列になっている
  - 『鉄拳8』ではキャラクターは近距離にしか出てこないため LOD は 0 しか使っていない

```
FSkeletalMeshLODRenderData& LODRenderData =
                    pSkeletalMesh->GetResourceForRendering()-> LODRenderData[0];
```

FSkeletalMeshLODRenderData クラス

- 重要なメンバーのみ抜粋

```
class FSkeletalMeshLODRenderData : public FRefCountBase
{
public:

        TArray<FSkelMeshRenderSection>          RenderSections;

        FMultiSizeIndexContainer            MultiSizeIndexContainer;

        FStaticMeshVertexBuffers            StaticVertexBuffers;

        FSkinWeightVertexBuffer             SkinWeightVertexBuffer;
```

FSkeletalMeshLODRenderData クラス

- 分割されているセクションを合わせる

```
class FSkeletalMeshLODRenderData : public FRefCountBase
{
public:

        TArray<FSkelMeshRenderSection>          RenderSections;

        FMultiSizeIndexContainer            MultiSizeIndexContainer;

        FStaticMeshVertexBuffers            StaticVertexBuffers;

        FSkinWeightVertexBuffer             SkinWeightVertexBuffer;
```

マテリアルごと
一定ボーン数ごとに
頂点バッファ
インデックスバッファ
が分かれている

FSkeletalMeshLODRenderData クラス

- インデックスバッファは特に使わないのでコピーするだけ

```
class FSkeletalMeshLODRenderData : public FRefCountBase
{
public:

        TArray<FSkelMeshRenderSection>          RenderSections;

        FMultiSizeIndexContainer        MultiSizeIndexContainer;

        FStaticMeshVertexBuffers        StaticVertexBuffers;

        FSkinWeightVertexBuffer         SkinWeightVertexBuffer;
```

16bit/32bit 両対応の
インデックスバッファ

FSkeletalMeshLODRenderData クラス

- position と normal は変換、UV はコピー

```
class FSkeletalMeshLODRenderData : public FRefCountBase
{
public:

        TArray<FSkelMeshRenderSection>          RenderSections;

        FMultiSizeIndexContainer        MultiSizeIndexContainer;

        FStaticMeshVertexBuffers        StaticVertexBuffers;

        FSkinWeightVertexBuffer         SkinWeightVertexBuffer;
```

position
UV
normal
color などの
頂点データ

FSkeletalMeshLODRenderData クラス

- ボーンとウェイトの情報

```
class FSkeletalMeshLODRenderData : public FRefCountBase
{
public:

        TArray<FSkelMeshRenderSection>          RenderSections;

        FMultiSizeIndexContainer        MultiSizeIndexContainer;

        FStaticMeshVertexBuffers        StaticVertexBuffers;

        FSkinWeightVertexBuffer         SkinWeightVertexBuffer;
```

インフルエンスのある
ボーンとそのウェイト
セクションごとに
ボーンの変換が必要

FSkelMeshRenderSection クラス

- 重要なメンバーのみ抜粋

pSkeletalMesh->GetMaterials()[]
マテリアルへのインデックス
LOD を使っているともう少し複雑だが
今回は割愛

```
struct FSkelMeshRenderSection
{
        uint16 MaterialIndex;

        uint32 BaseIndex;
        uint32 BaseVertexIndex;

        TArray<FBoneIndexType> BoneMap;
        // typedef uint16 FBoneIndexType;
```

FSkelMeshRenderSection クラス

- 重要なメンバーのみ抜粋

```
struct FSkelMeshRenderSection
{
        uint16 MaterialIndex;

        uint32 BaseIndex;
        uint32 BaseVertexIndex;

        TArray<FBoneIndexType> BoneMap;
        // typedef uint16 FBoneIndexType;
```

それぞれ
インデックスバッファ
頂点バッファの先頭への
インデックスオフセット

FSkelMeshRenderSection クラス

- 重要なメンバーのみ抜粋
- BoneMap については次ページにて

```
struct FSkelMeshRenderSection
{
        uint16 MaterialIndex;

        uint32 BaseIndex;
        uint32 BaseVertexIndex;

        TArray<FBoneIndexType> BoneMap;
        // typedef uint16 FBoneIndexType;
```

スケルタルメッシュ全体の
マトリクス列(ボーン列)と
セクションローカルの
マトリクス列の対応マップ

セクションローカルのマトリクス列へ変換

- セクションごとに頂点をマトリクスで変換すればできあがり

メッシュ変形前後のおまじない

- GPU リソースを書き換えるときはアクセス競合しないように
  - 変形前にリソース解放
  - 変形後にリソース初期化

```
// 変形前.
pSkeletalMesh->ReleaseResources();
pSkeletalMesh->ReleaseResourcesFence.Wait();

// 変形後.
TUniquePtr<FSkinnedMeshComponentRecreateRenderStateContext>
        RecreateRenderStateContext =
        MakeUnique<FSkinnedMeshComponentRecreateRenderStateContext>(pSkeletalMesh);
pSkeletalMesh ->InitResources();
```

問題点

頂点数不一致

- プロポメッシュとマッスルメッシュの頂点数が合わない
- インポート時に頂点列とインデックスが一度展開される
- このとき同一の値を持つ頂点は統合される
  - 最適化してある FBX でもこれが起こる
  - ここをスキップすることはできない
- ただし同じアルゴリズムのため毎回同じ結果になる
- そこで統合されないように頂点カラーを入れてもらうことで回避

ウェイトの精度不足

- Maya 上と同じ体型にならない問題
- SkeletalMesh のウェイトが 8 ビット整数(0～255) で精度不足
  - インポート時の量子化に癖があり
- Maya 側でインポート時と同じ量子化を行うツールを作成し調整
- UE 5.2 にて 16 ビット整数になるが時すでに遅し

DuplicateObject 問題

- USkeletalMesh はランタイムで作るようにはできていない
- ロードした USkeletalMesh を編集すると
  オリジナルが書き換わってしまう！
- ロードした USkeletalMesh を
  複製して書き換えることにしたが・・・

```
USkeletalMesh* pNewMesh =
            static_cast<USkeletalMesh*>(StaticDuplicateObject(pOrgMesh, …));
```

DuplicateObject 問題

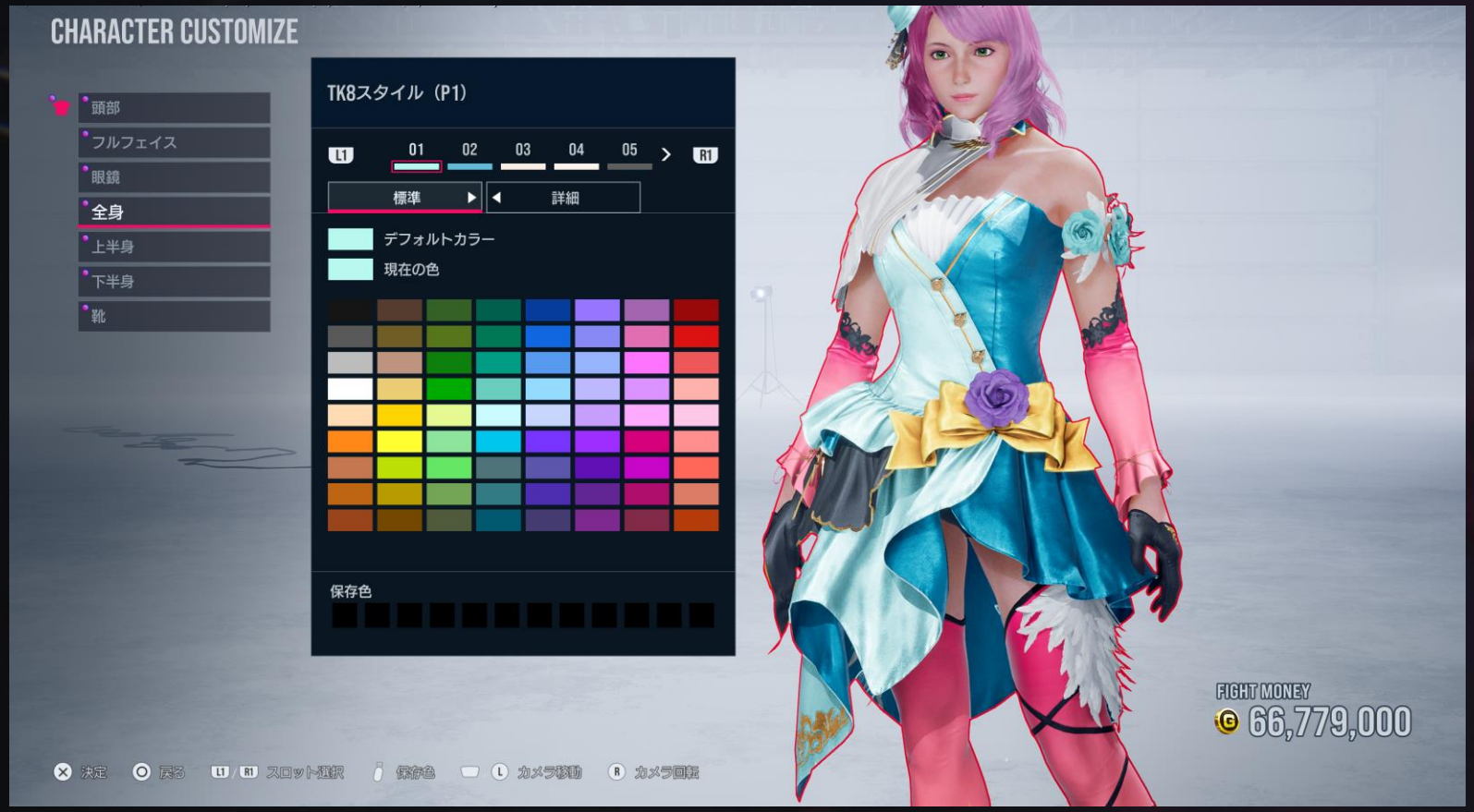
- StaticDuplicateObject() は非常に遅い
  - シリアライズ・デシリアライズを行っているため
- 結果、体格調整の処理時間の大半が StaticDuplicateObject() ということに！
- Epic のかたから自分でコピーすることを勧めていただくも
- バージョン違いの吸収、コンポーネントもコピーしてくれる StaticDuplicateObject() に頼ることに
- シリアライズ・デシリアライズの仕組みは偉大

その他の事例

テクスチャーベイク

ここで使用！

- このような色カスタマイズ

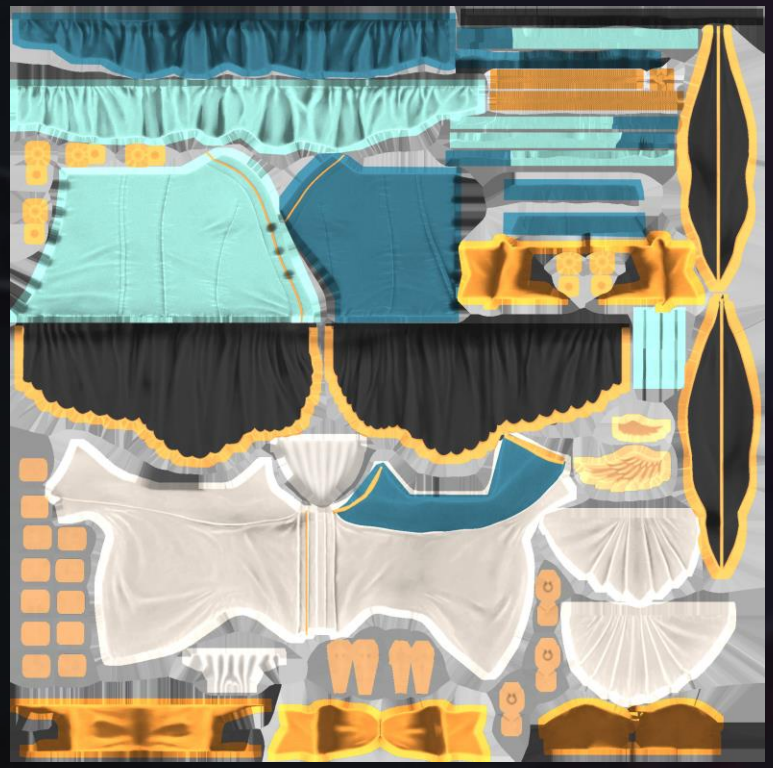


なぜテクスチャーベイク

- 『鉄拳7』では色カスタマイズはマテリアル内で計算し表現
  - そのため制限があった
- 『鉄拳8』になりカスタマイズ性を高めるため
  毎回同じ計算結果が出るカスタマイズ表現は
  結果をあらかじめテクスチャーをベイクして生成することにした

色ベイクの実例

- アリサのデフォルトコスチューム



カラーマップ以外も対応

- BaseColor
  - 色カスタマイズ, テクスチャー合成, フェイスペイント, メイク, 日焼け
- Normal
  - 絆創膏など
  - ちゃんとベクトルにデコードし合成した後エンコードする
- その他
  - Roughness,Metalness,AmbientOcclusion,Metalness,Translucency,Mask など様々なマップに対応

問題点

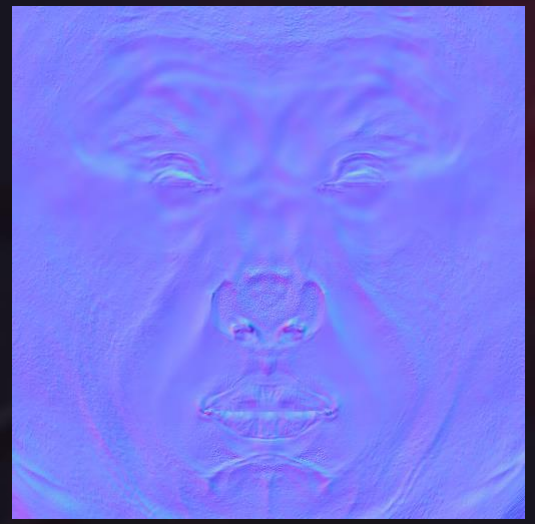
- 処理時間がかかる
  - GPU でレンダリングするため
  - ベイクのコードは BP で書かれている
  - すべてのレンダリングが終わるまで画面が固まる
- CPU がブロックしないよう Vsync 時間ごとに処理するようにした
  - DrawMaterialToRenderTarget を独自のものを実装、そちらを呼び出す
  - BP は一気に実行
  - キューに溜めてブロックしないよう分割して
    本物の DrawMaterialToRenderTarget を呼び出すことで解決

しわ自動計算

しわ自動計算とは？

- 「しわ」あり Normal Map と「しわ」なし Normal Map
- 表情に応じて部分的にブレンドしてしわができるようにする
- そこでブレンド率を自動計算するコンピュートシェーダーを作成

しわあり



しわなし

処理手順

1. 顔の頂点バッファを自分でスキニング
2. インデックスバッファを読んでトライアングルを作成
3. スキニング前後のトライアングルの面積を比較
4. 結果をテクスチャー座標(UV)系でレンダーターゲットに描画

1〜3 はコンピュートシェーダー
1, 2, 3 は依存関係があるのでシーケンシャル実行だが
それぞれのディスパッチの中身は並列実行のため高速

4 は UV 値と 3 の結果を入力とした VS・PS

面積の求め方

- 面積 S はヘロンの公式より

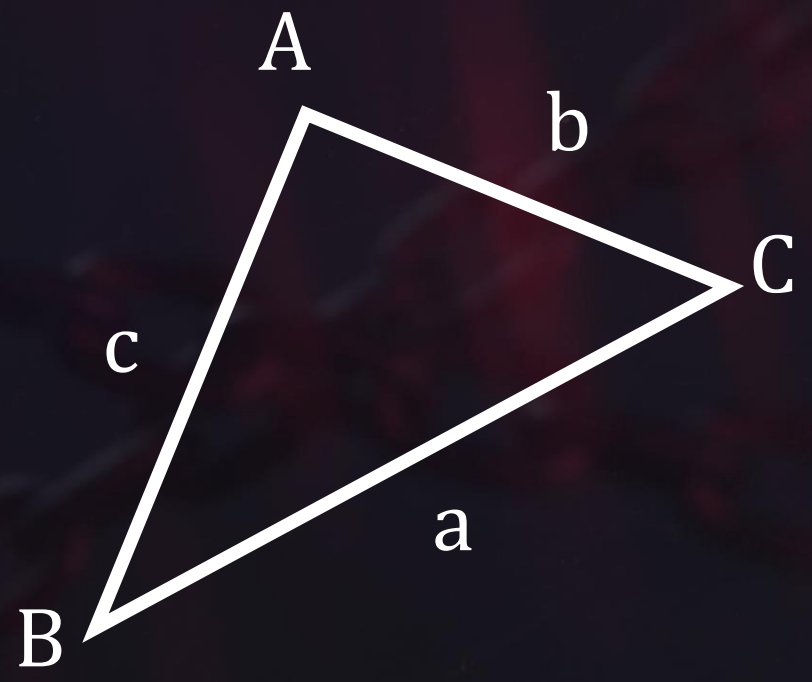
$$S = \sqrt{s(s-a)(s-b)(s-c)}$$

$$where\ s = \frac{a+b+c}{2}$$

$$a = length(C - B)$$

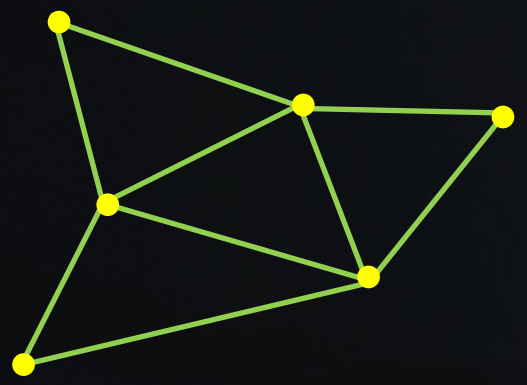
$$b = length(A - C)$$

$$c = length(B - A)$$



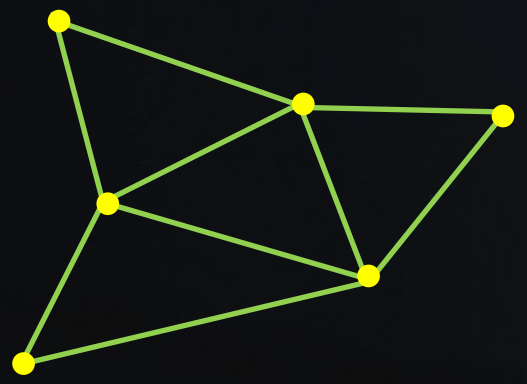
面積を比較

面積が拡大 → 皮膚が伸びる → しわが減る



面積を比較

面積が縮小 → 皮膚が縮む → しわができる



結果

- 緑が拡大傾向にあるところ、赤が縮小傾向にあるところ

まとめ

まとめ

- 今回キャラクター数×カスタマイズ数に対処する体格調整を実装
- エンジン改造することなく公開クラスを使って実装できた
- 同様にテクスチャーベイク、しわ自動計算も実装できた
- エンジンと仲良くやっていくことは可能
- ただネット上に情報が少ない(特に C++ やシェーダー)
- 一緒に情報発信していきましょう！

ご清聴ありがとうございました